

Code review — Inițiere WindowsForms

Runde de review pe proiectul tău, cea mai nouă sus.

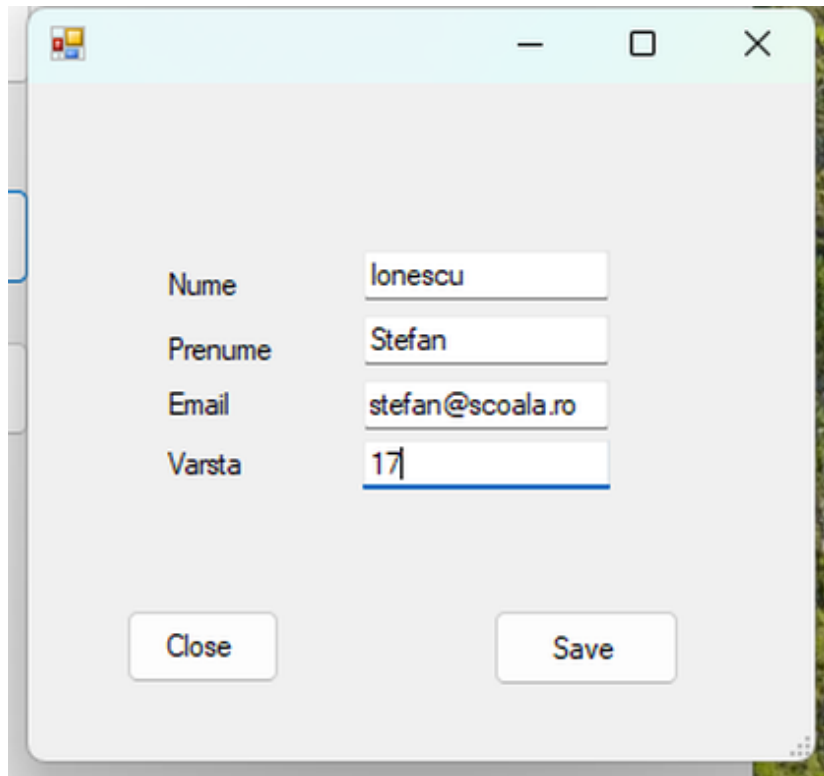
Runda 2 — 2026-09-23 — sha 4568aa1

Ai pushat la 18:16, la șapte minute după ce a plecat runda 1 — deci ai lucrat în paralel, fără s-o vezi. Constatările de acolo (B1, B2, B3) sunt încă deschise; le găsești mai jos, în runda 1.

Runda asta e pe codul nou: `Student.cs`, formularul de date, `Adapter`. Am compilat pe `net472` în Windows real, am pornit aplicația și **am completat formularul de la tastatură**, exact cum cerea fișa. Capturile sunt din execuție.

Ce merge

- compilează, cu 2 avertismente (vezi C1, C2)
- `Student.cs` e bine gândit — validarea stă în setter, nu în formular. Email-ul refolosește regula din `academy`, exact cum cerea fișa
- `TabIndex` 0–9 pus corect: nume → prenume → email → vârstă → Save → Close
- aspectul formularului e ordonat: etichete aliniate, casete aliniate, butoane jos
- feedback vizual pe câmpul greșit (fond roșu)



Formularul completat de la tastatură

● Critice

B1 — Un Tab din câmpul Vârstă închide formularul

`Form1.Designer.cs:117` , `Form1.cs:56-60`

CUM E ACUM — reprodus prin rulare: am completat de la tastatură `Ionescu` → `Stefan` → `stefan@scoala.ro` → `17` , apoi am apăsas **Tab o singură dată**. Formularul s-a închis. Fără click pe Save, fără Enter, fără să apăs nimic altceva.

DE CE se întâmplă: ai legat `Form1_Enter` la evenimentul `Enter` al butonului Save:

```
this.btnSave.Enter += new System.EventHandler(this.Form1_Enter);
```

`Enter` , la un control, **nu are nicio legătură cu tasta Enter**. Înseamnă „controlul tocmai a primit focus”. Iar focusul ajunge pe Save exact atunci când navighezi cu Tab — adică fix comportamentul pe care fișa ți-l cerea la exercițiul 4 („Tab trebuie sa mearga nume → email → varsta → Salveaza”).

Deci ai construit corect ordinea de Tab și ai pus pe ultimul pas o metodă care închide fereastra. Cele două cerințe se anulează reciproc.

Mai e o legătură, pe formular: `this.Enter += Form1_Enter ( Form1.Designer.cs:143 )`. Aceea e inertă — un `Form` primește `Activated`, nu `Enter` — dar arată că ai căutat evenimentul „s-a apăsător Enter” și ai găsit unul cu același nume și alt înțeles.

FIX: pentru „Enter apasă Salvează” și „Esc apasă Renunță” nu se folosesc evenimente, ci două proprietăți ale formularului:

```
this.AcceptButton = this.btnSave;  
this.CancelButton = this.btnClose;
```

Apoi muți logica de salvare în `btnSave.Click`, iar legăturile pe `Enter` le ștergi din Properties → Events.

B2 — Datele completate nu ajung înapoi în `App`

`Form1.cs:58`, `App.cs:39-43`

CUM E ACUM: `Form1_Enter` face `student = aux;` — iar în `App`, după `ShowDialog()`, obiectul `student` e neschimbat. Ai bănuir asta singur: ai pus `Console.WriteLine(student.Varsta)` la `App.cs:43` ca să verifici. Într-o aplicație WinForms nu există consolă, deci linia aia nu afișează nimic, nicăieri. De-aia n-ai văzut problema.

DE CE se întâmplă: `student` e un **câmp al formei**, inițializat în constructor cu referința primită din `App`. Când scrii `student = aux`, nu modifici obiectul — muți câmpul să arate spre alt obiect. `App` păstrează referința lui originală, care n-a fost atinsă.

E aceeași distincție pe care o știi din C#: a modifica obiectul prin referință (`student.Varsta = 17`) versus a înlocui referința (`student = aux`). Prima se vede afară, a doua nu.

Și de aici iese ceva mai neplăcut: formularul se închide, utilizatorul crede că a salvat, iar `App` rămâne cu un `Student` gol. Un bug care nu dă nicio eroare.

FIX: ori copiezi valorile în obiectul primit:

```
student.Nume = txtNume.Text;  
student.Email = aux.Email;  
student.Varsta = aux.Varsta;
```

ori expui rezultatul ca proprietate publică și-l citești din `App` după `ShowDialog()` :

```
// in Form1
public Student Rezultat { get; private set; }
// in App
if (form1.ShowDialog() == DialogResult.OK) { student = form1.Rezultat; }
```

A doua variantă e cea folosită în practică, pentru că merge mână în mână cu `DialogResult`.

B3 — `TryParse` chemat, dar rezultatul lui ignorat

Form1.cs:37-39

CUM E ACUM:

```
int nr;
Int32.TryParse(txtVarsta.Text, out nr);
aux.Varsta = nr;
```

Scrie „abc” în câmpul vârstă și primești mesajul „**Varsta gresita**”.

DE CE e greșit mesajul: `TryParse` nu aruncă excepție — asta e tot rostul lui. Întoarce `bool` : `true` dacă a reușit, `false` dacă nu, iar la eșec pune `0` în variabila de ieșire. Tu ignori valoarea întoarsă, deci la text invalid `nr` rămâne `0`, iar `aux.Varsta = 0` intră în setterul tău și aruncă `ArgumentException("Varsta gresita")`.

Utilizatorul citește că vârsta e greșită. Dar vârsta nu e greșită — **nu e un număr**. Mesajul descrie a doua eroare, nu prima. Fișa spunea „un mesaj care spune exact ce camp e gresit”; aici nici măcar motivul nu e cel real.

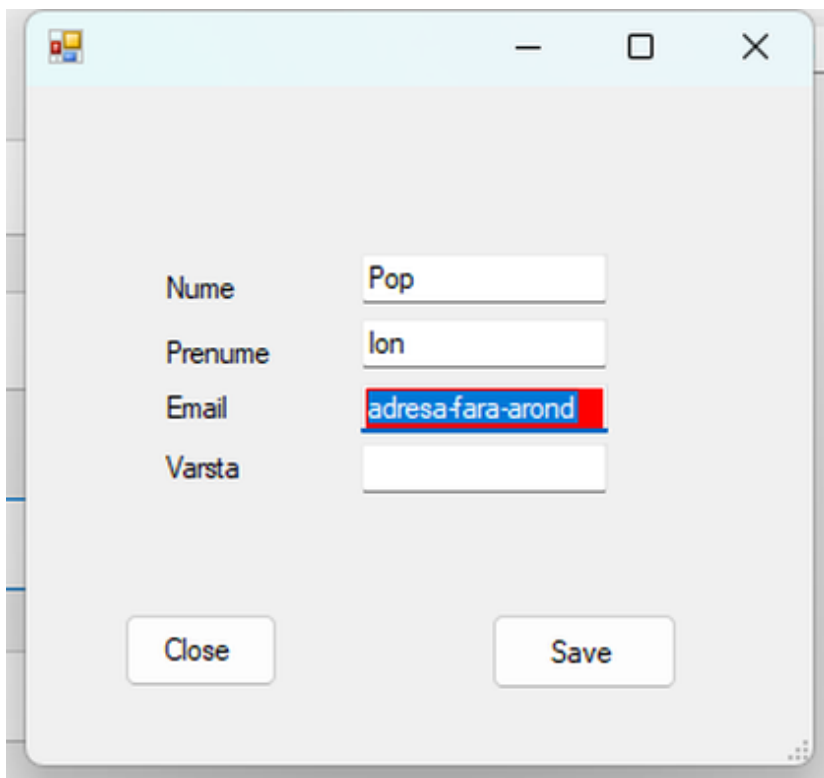
FIX:

```
int nr;
if (!Int32.TryParse(txtVarsta.Text, out nr))
{
    MessageBox.Show("Varsta trebuie sa fie un numar.");
    txtVarsta.SelectAll();
    return;
}
aux.Varsta = nr;
```

B4 — Câmpul Email te ține captiv, fără să-ți spună de ce

Form1.cs:25-30

CUM E ACUM — reprodus prin rulare: am scris `adresa-fara-arond` în Email și am apăsă **Tab de două ori**. Cursorul a rămas în Email. Câmpul Vârstă a rămas gol — n-am putut ajunge la el:

A screenshot of a Windows application window titled "Form1". The window contains a form with four text input fields: "Nume" (containing "Pop"), "Prenume" (containing "Ion"), "Email" (containing "adresa-fara-arond"), and "Varsta" (empty). The "Email" field is highlighted with a red border. At the bottom of the form are two buttons: "Close" and "Save". The window has standard Windows window controls (minimize, maximize, close) in the title bar.

Email invalid: Tab apăsă de două ori, cursorul tot acolo

DE CE se întâmplă: `Leave` se declanșează **în momentul în care pleci** din control. Tu, în handler, cheme `txtEmail.Focus()` — adică te întorci. Pleci, ești adus înapoi, pleci din nou, ești adus înapoi. Cât timp textul e invalid, câmpul nu-ți mai dă drumul.

Fișa cerea „lasi cursorul în câmpul acela” — dar în handlerul butonului Salvează, nu la ieșirea din câmp. Diferența e că acolo utilizatorul a cerut salvarea; aici e prins fără să fi cerut nimic.

Mai rău: `catch (ArgumentException ex)` de la email **nu afișează niciun mesaj** (de-aia și avertismentul `CS0168`, vezi C2). Utilizatorul e blocat într-un câmp roșu și nu i se spune ce vrei de la el. La vârstă ai pus `MessageBox.Show(ex.Message)`; la email, nu. Aceeași situație, două comportamente.

FIX: validarea la `Leave` doar colorează și afișează mesajul — fără `Focus()`. Blocarea cursorului se face în `btnSave.Click`, când chiar se încearcă salvarea.

🟡 Importante

M1 — Butonul Close nu răspunde la click

`Form1.Designer.cs:127`, `Form1.cs:51-54`

```
this.btnClose.KeyPress += new KeyPressEventHandler(this.btnClose_KeyPress);
```

`KeyPress` se declanșează când butonul are focus **și apeși o tastă**. Un click cu mouse-ul nu produce `KeyPress`. Deci butonul Close se poate apăsa cu mouse-ul și nu se întâmplă nimic — și, cum ai văzut la B1, formularul se închide în schimb când ajungi cu Tab pe Save. Exact invers decât ar trebui.

Evenimentul corect e `Click` — iar pentru Esc, `CancelButton` (vezi B1).

M2 — **Adapter** e o formă goală cu un buton fără cod

`Adapter.cs:20-23`, `App.cs:63-68`

Forma se deschide din meniu, nu conține nimic, iar `button1_Click` are corpul gol. Dacă e început de exercițiu, lasă-l; dacă e rămas dintr-o încercare, șterge-l — la concurs o formă goală care se deschide din meniu arată ca o cerință nerezolvată.

Separat: numele `Adapter` vine din design patterns. Aici n-are legătură cu acel Adapter. La opt exerciții, numele contează — `Ex5Form`, `Ex6Form` îți spun imediat unde ești.

M3 — Formularul n-are titlu

`Form1.Designer.cs:131-143`

În capturi, bara de titlu e goală. `Text` nu e setat, la fel cum nu erau setate `StartPosition`, `FormBorderStyle`, `MaximizeBox` — aceeași constatare ca B2 din runda 1, acum și pe forma nouă.

Un formular fără titlu la olimpiadă costă din cele 12 puncte de „cromatică și design”, pentru că e primul lucru care se vede.

● Cleanups

C1 — `button1` declarat și niciodată folosit. `Form1.Designer.cs:151` — avertisment `CS0169` la compilare. A rămas de la forma veche, când `Form1` avea butonul „ce mai faci”.

C2 — `ex` prins și nefolosit. `Form1.cs:25` — avertisment `CS0168`. Consecința funcțională e la B4: excepția e prinsă și aruncată la gunoi, fără mesaj.

C3 — handler-e goale rămase. `App.cs:21–29` (`textBox1_TextChanged`, `App_Load`), `App.cs:46–49` (`lbl_Click`), `App.cs:58–61` (`toolStripMenuItem1_Click`, cu cod comentat înăuntru). Șterge-le din Properties → Events, nu doar din cod.

C4 — `lbl_MouseEnter` afișează un `MessageBox`. `App.cs:51–56` — treci cu mouse-ul peste etichetă și primești o fereastră de dialog. Dacă a fost experiment, scoate-l; e genul de lucru care la concurs blochează evaluatorul.

Before / After

B1 — Enter și Esc

Acum	Cum trebuie
<pre>this.btnSave.Enter += Form1_Enter; this.Enter += Form1_Enter;</pre>	<pre>this.AcceptButton = this.btnSave; this.CancelButton = this.btnClose; this.btnSave.Click += btnSave_Click;</pre>

B3 — parsarea vârstei

Acum	Cum trebuie
<pre>int nr; Int32.TryParse(txtVarsta.Text, out nr); aux.Varsta = nr;</pre>	<pre>int nr; if (!Int32.TryParse(txtVarsta.Text, out nr)) { MessageBox.Show("Varsta trebuie sa fie un numar."); return; } aux.Varsta = nr;</pre>

B4 — validarea la ieșirea din câmp

Acum	Cum trebuie
<pre>catch (ArgumentException ex) { txtEmail.BackColor = Color.Red; txtEmail.SelectAll(); txtEmail.Focus(); }</pre>	<pre>catch (ArgumentException ex) { txtEmail.BackColor = Color.Red; lblEroare.Text = ex.Message; }</pre>

Q&A — răspunde în scris

1. `btnSave.Enter` și tasta Enter n-au nicio legătură. Cum se numește, atunci, evenimentul care se declanșează când utilizatorul apasă efectiv tasta Enter într-un formular — și de ce fișa îți cerea `AcceptButton` în loc de orice eveniment?
2. În `Form1_Enter` scrii `student = aux`. Dacă în loc de asta ai scrie `student.Varsta = aux.Varsta`, s-ar vedea schimbarea în `App`? De ce una da și cealaltă nu, dacă `student` e același câmp în ambele cazuri?
3. `Int32.TryParse` are tipul de retur `bool`. La ce folosește valoarea aia, dacă metoda oricum pune rezultatul în `out nr`? Ce valoare are `nr` după un `TryParse` eșuat, și de ce tocmai aia?

Runda 1 — 2026-09-23 — sha e7a0bd0

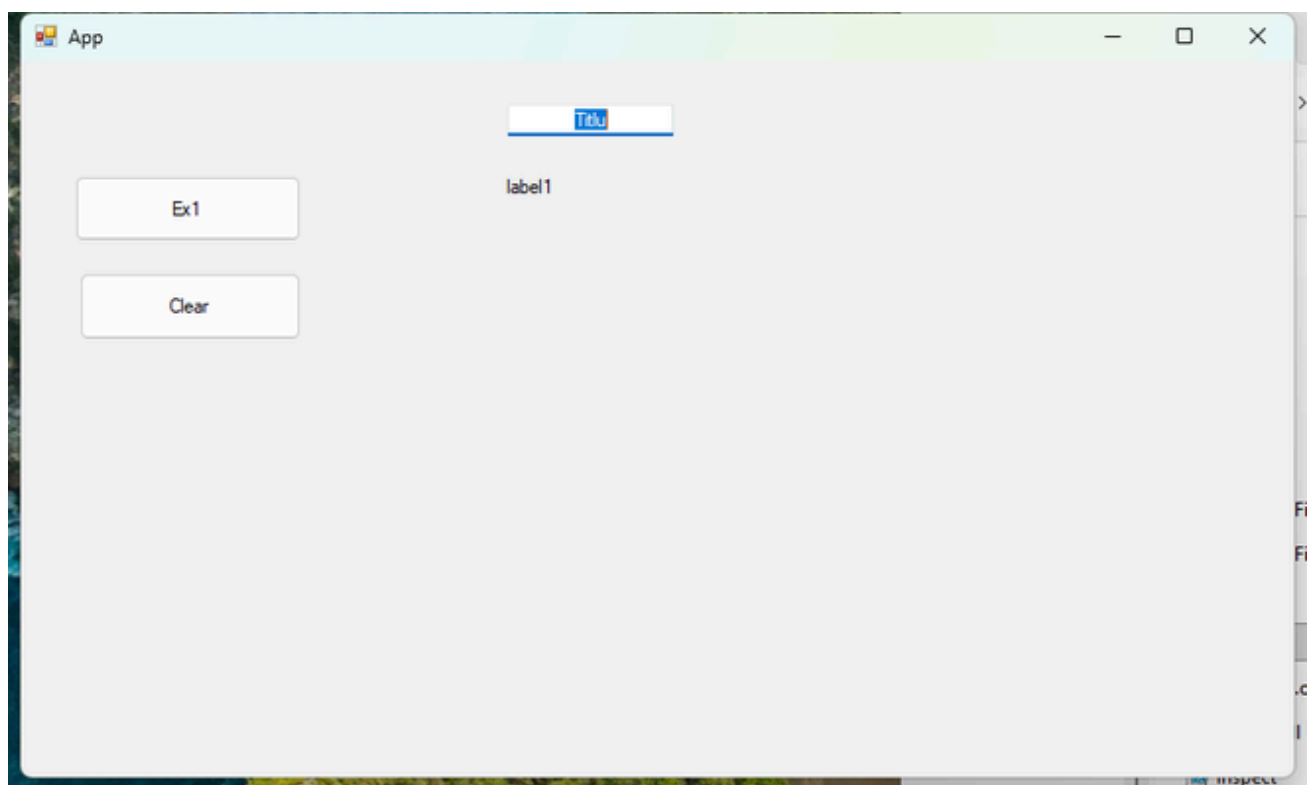
Prima rundă pe WinForms. De data asta review-ul s-a făcut **rulând aplicația**, nu citind codul: proiectul a fost compilat pe .NET Framework 4.7.2 într-un Windows real și pornit, iar capturile de mai jos sunt din execuția lui, nu mockup-uri.

Ce merge

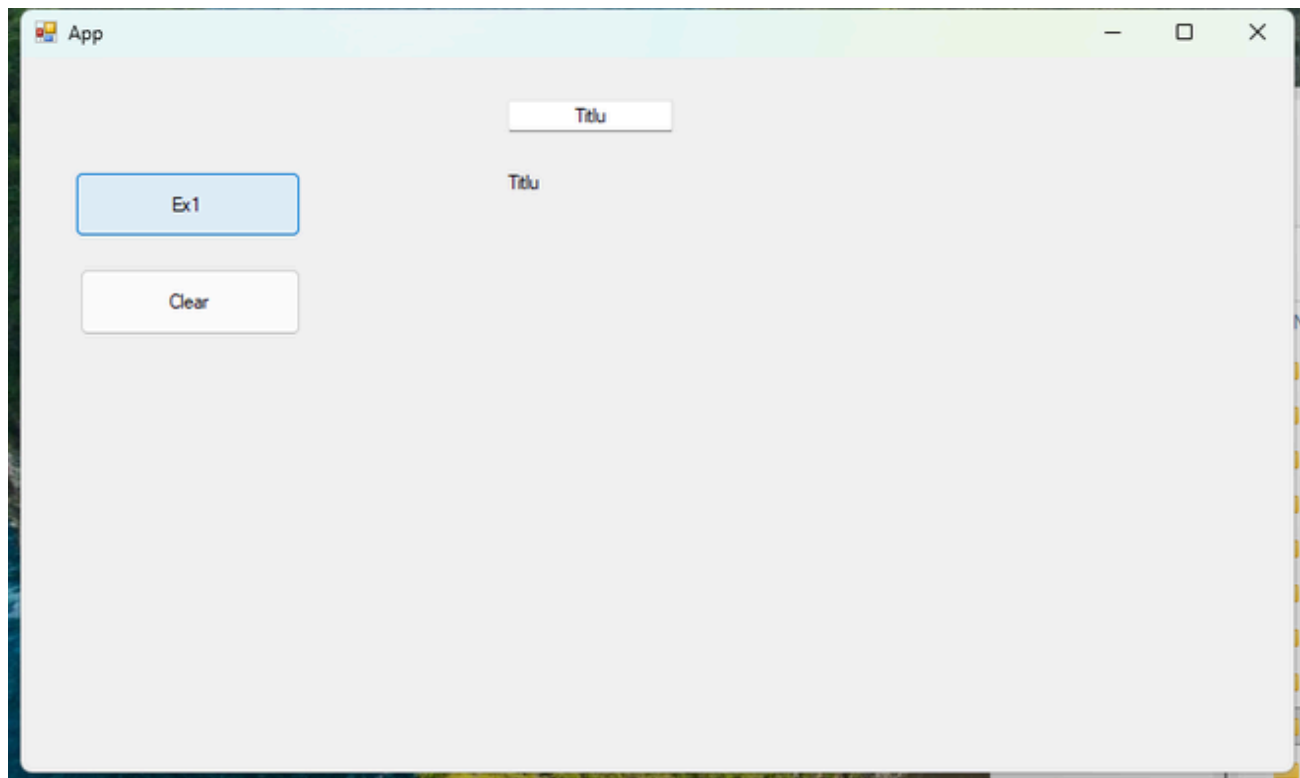
Verificat prin rulare, nu prin citire:

- proiectul **compilează** pe `net472`, fără erori și fără avertismente
- `Program.cs` pornește `App`, nu `Form1` — exact cum cerea exercițiul 2
- butonul **Ex1** copiază textul din casetă în etichetă
- butonul **Clear** golește și caseta, și eticheta

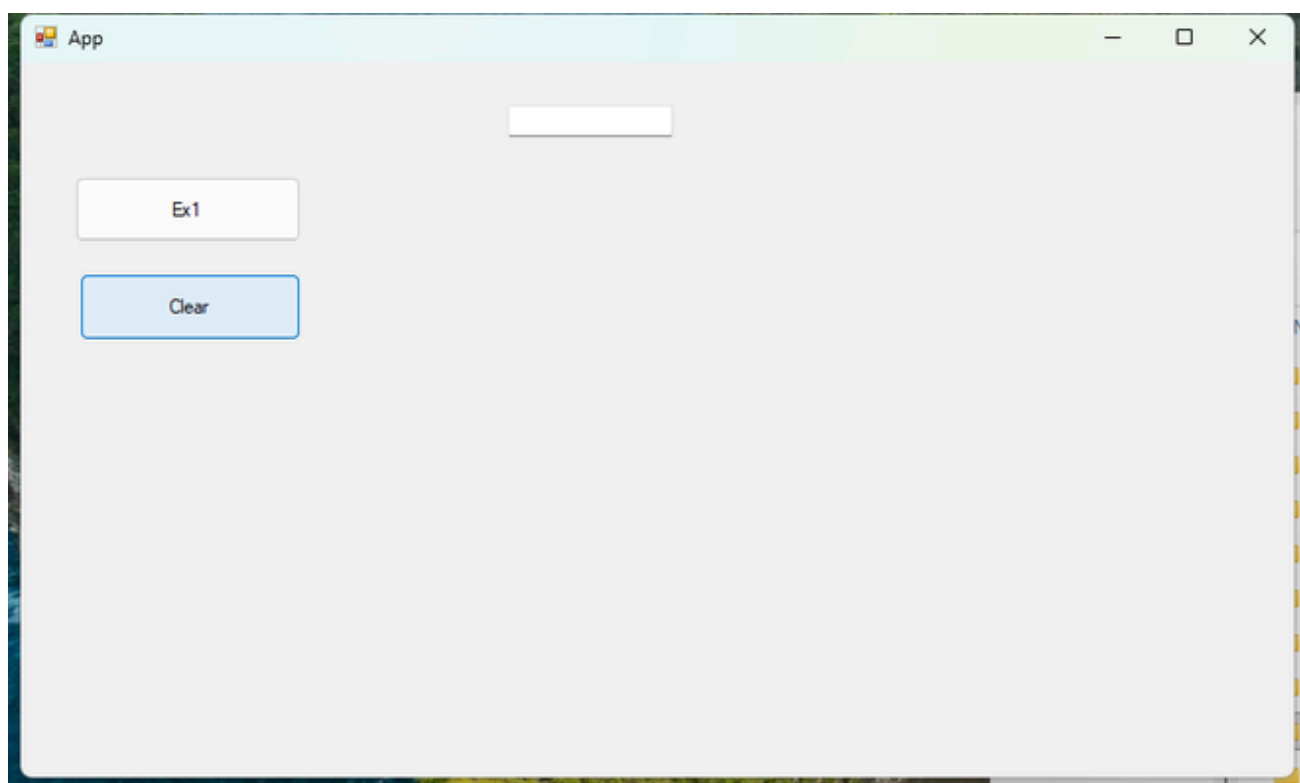
Logica pe care ai scris-o funcționează. Tot ce urmează e despre *unde* ai pus-o și despre ce a rămas nefăcut din fișă.



Aplicația la pornire



După apăsarea butonului Ex1



După apăsarea butonului Clear

Critice

B1 — Exercițiul 3 e construit în formularul-meniu, nu în formularul lui

`App.Designer.cs:31-34` , `App.Designer.cs:81-84`

CUM E ACUM: caseta de text, eticheta și cele două butoane stau direct pe forma `App` . `App` era, conform fișei, meniul aplicației — locul din care deschizi exercițiile 2–8.

DE CE contează: fișa spune „Un exercitiu = o forma” și cere `Ex3Form.cs` . Nu e o preferință de organizare, e felul în care WinForms te obligă să lucrezi. O formă e o clasă: toate controalele ei devin câmpuri ale aceleiași clase, iar toate handler-ele devin metode ale aceleiași clase.

Când ajungi la exercițiul 4 (formularul cu nume, email, vârstă) și-l pui tot în `App` , ai în aceeași clasă două casete `txtNume` și `txtTilu` care n-au nicio legătură între ele, plus handler-ele amândurora. La exercițiul 8 ai o clasă cu 30 de câmpuri și 20 de metode, în care nu mai găsești nimic — și, mai rău, Designer-ul îți desenează toate controalele suprapuse pe aceeași suprafață.

E exact problema pe care ai avut-o în `academy` cu clasele care făceau prea multe. Aici se vede mai repede, pentru că o vezi cu ochii.

FIX: mută cele patru controale într-o formă nouă `Ex3Form` , iar în `App` pune un buton `btnEx3` care face:

```
Ex3Form f = new Ex3Form();  
f.Show();
```

B2 — Exercițiul 2 e nefăcut: fereastra n-are niciuna din proprietățile cerute

`App.Designer.cs:76-87`

CUM E ACUM: verificat pe aplicația pornită:

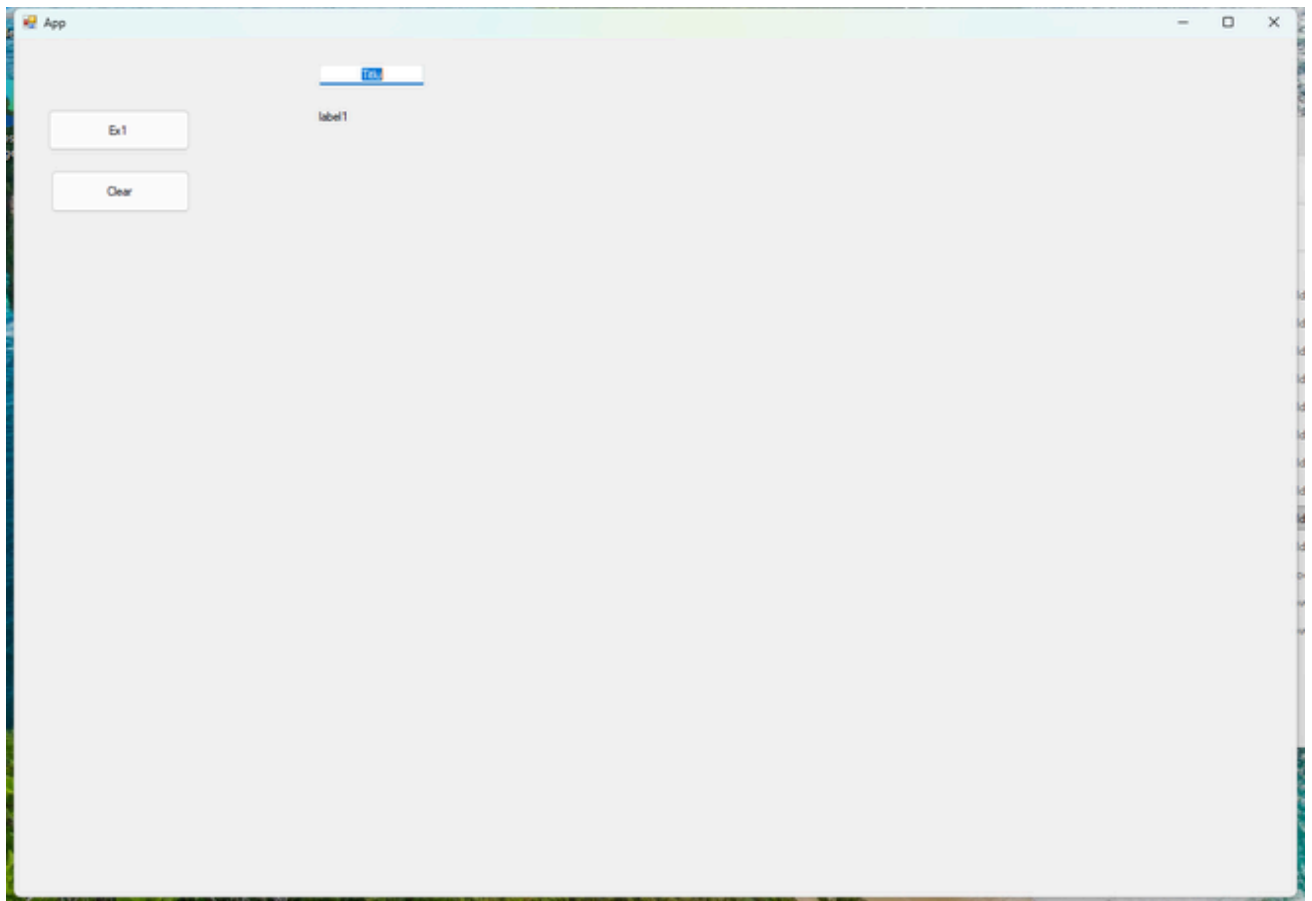
```
redimensionabila : True  
maximize activ    : True  
pozitie la start  : 156,156   (ecran 1919x1027)
```

Fișa cerea explicit: fereastra **să nu poată fi redimensionată** și **să apară pe centrul ecranului**, prin `FormBorderStyle` , `MaximizeBox` , `StartPosition` , `Text` . Niciuna dintre cele patru nu apare în

Designer.

DE CE se întâmplă: o proprietate pe care n-o atingi în fereastra Properties **nu apare deloc** în `InitializeComponent()`. Designer-ul scrie doar ce diferă de valoarea implicită. Iar implicitele sunt `FormBorderStyle = Sizable`, `MaximizeBox = true`, `StartPosition = WindowsDefaultLocation`.

Deci absența liniilor nu înseamnă „n-am apucat”, înseamnă „am lăsat comportamentul implicit” — și comportamentul implicit e fix cel pe care fișa îți cerea să-l schimbi. De-aia fereastra ta poate fi trasă la orice dimensiune, iar când o tragi, controalele rămân îngrămădite în colț:



Aceeași fereastră, trasă la 1300x900

Asta nu e o chestiune de cosmetică. În baremele reale de olimpiadă, **„cromatică și design” valorează 12 puncte din 100**, iar „portabilitate” încă 5. Sunt mai multe puncte decât cele mai multe cerințe de logică luate individual.

FIX: în fereastra Properties a formei `App`:

```
FormBorderStyle = FixedSingle
MaximizeBox      = False
StartPosition    = CenterScreen
Text             = Exercitii WinForms - Stefan
```

B3 — Textele puse de Designer rămân vizibile la pornire

`App.Designer.cs:74` (`lbl.Text = "label1"`), `App.Designer.cs:43` (`txtTilu.Text = "Titlu"`)

CUM E ACUM: în prima captură de mai sus, utilizatorul vede scris `label1` sub casetă. Nu e un text pe care l-ai ales tu; e numele implicit pe care Designer-ul îl pune în `Text` când tragi un `Label` pe formă.

DE CE e o capcană, nu doar o scăpare: la un `Label`, Designer-ul inițializează `Text` cu numele controlului. Dacă nu-l ștergi, el rămâne în cod ca orice altă proprietate și se afișează la pornire. Aceeași poveste la `TextBox`, unde tu ai lăsat `"Titlu"`.

Și de aici iese ceva mai subtil: dacă pornești aplicația și apeși **Ex1** direct, fără să scrii nimic, eticheta afișează `Titlu` — și pare că totul funcționează. De fapt tocmai ai copiat textul de placeholder dintr-un control în altul. Un bug de logică ascuns sub o valoare implicită arată exact așa: aplicația „merge”, dar nu din motivul pe care-l crezi.

FIX: `lbl.Text` gol, iar în `txtTilu` fie gol, fie un `PlaceholderText` adevărat (proprietatea există în .NET Framework doar de la 4.7.2 în sus — o ai).

Importante

M1 — Numele controalelor și ale handler-elor nu se potrivesc între ele

`App.Designer.cs:31-34`, `App.cs:20`, `App.cs:25`, `App.cs:35`

Patru nepotriviri în aceeași formă:

Control	Nume	Problema
TextBox	txtTilu	scris greșit — „Titlu”
Button (Clear)	button2	nume implicit, deși are <code>Text = "Clear"</code>
Button (Ex 1)	Ex1	conține de fapt exercițiul 3
handler	button1_Click	butonul se numește <code>Ex1</code> , nu <code>button1</code>
handler	textBox1_TextChanged	caseta se numește <code>txtTilu</code> , nu <code>textBox1</code>

Handler-ele păstrează numele pe care controlul îl avea **în momentul dublu-click-ului**. Dacă redenumești controlul după aceea, Designer-ul actualizează legătura, dar nu și numele metodei — de aceea ajungi cu `Ex1.Click += button1_Click`. Peste opt exerciții, asta devine imposibil de citit.

Redenumește controlul **înainte** de a-i scrie handler-ul, iar numele metodei îl obții gratis.

M2 — `Form1` a rămas în proiect ca schiță de test

`Form1.cs:21-26`, `Form1.Designer.cs:31-42`

Un buton cu textul „ce mai faci” care afișează un `MessageBox`. Forma nu e deschisă de nicăieri — `Program.cs` pornește `App`. E cod care se compilează, se livrează și nu se execută niciodată.

Fișa îți cerea s-o deschizi ca să înțelegi `partial` și `Designer.cs`; asta ai făcut. Acum ori o transformi în `Ex3Form` (are deja infrastructura), ori o ștergi din proiect.

Cleanups

C1 — Handler-e goale, legate în Designer. `App.cs:25-33` (`textBox1_TextChanged`, `App_Load`), `Form1.cs:28-31` (`Form1_Load`). Un dublu-click accidental pe control creează handler-ul și legătura. Metoda goală e inofensivă, dar linia din Designer rămâne — iar dacă ștergi metoda fără să ștergi legătura, proiectul nu mai compilează. Șterge-le pe amândouă din Properties → Events.

C2 — Exercițiul 1 n-are răspunsuri scrise nicăieri. Fișa cerea, în scris, ce face `Application.Run(...)`, cine scrie `Designer.cs` și de ce forma apare în două fișiere. Nu există nici

fișier, nici comentariu în cod. Întrebările nu erau umplutură — vezi Q&A la final.

C3 — `using` -uri nefolosite. `App.cs:2-8` , `Form1.cs:2-8` : `System.Linq` , `System.Threading.Tasks` , `System.Data` , `System.Text` nu sunt folosite în niciuna din forme. Vin din șablonul Visual Studio.

Before / After

B2 — proprietățile ferestrei

Acum (<code>App.Designer.cs:76-87</code>)	Cum trebuie
<pre>this.ClientSize = new Size(800, 450); this.Name = "App"; this.Text = "App";</pre>	<pre>this.ClientSize = new Size(800, 450); this.FormBorderStyle = FormBorderStyle.FixedSingle; this.MaximizeBox = false; this.StartPosition = FormStartPosition.CenterScreen; this.Name = "App"; this.Text = "Exercitii WinForms - Stefan";</pre>

B3 — textele de placeholder

Acum	Cum trebuie
<pre>this.lbl.Text = "label1"; this.txtTilu.Text = "Titlu";</pre>	<pre>this.lblRezultat.Text = ""; this.txtTitlu.Text = "";</pre>

B1 — structura

Acum	Cum trebuie
<pre>App</pre> conține <code>txtTilu</code> , <code>Ex1</code> , <code>button2</code> , <code>lbl</code>	<pre>App</pre> conține <code>btnEx3</code> <pre>Ex3Form</pre> conține <code>txtTitlu</code> , <code>btnAfiseaza</code> , <code>btnClear</code> , <code>lblRezultat</code>

Q&A — răspunde în scris

Sunt întrebările din exercițiul 1, la care n-ai răspuns, plus una din review:

1. `Application.Run(new App())` — ce face, de fapt? De ce programul nu se termină imediat după linia asta, așa cum s-ar termina un `Console.WriteLine` ? Ce anume ține procesul în viață?
2. De ce clasa `App` e scrisă în două fișiere (`App.cs` și `App.Designer.cs`) și ce cuvânt-cheie face asta posibil? Ce s-ar întâmpla dacă ai scrie `InitializeComponent()` de mână în `App.cs` ?
3. În `App.Designer.cs` **nu există** nicio linie despre `FormBorderStyle` . Fereastra ta e totuși redimensionabilă. De unde vine comportamentul ăsta, dacă nu e scris nicăieri în cod?



MyCodeSchool

Material didactic MyCodeSchool

mycodeschool.ro